

CS-334: Machine Learning

Lecture 10: Model Selection

(with In-Class Exercise #8)

09/29/2026

Prof. Shengpu Tang



Announcements

- HW2 due
 - solutions will be posted Wed
 - will try to grade by Thursday
- HW3 released
- HW2 + HW3 Extra Credit
 - due 10/19

Homework 3 - Predicting Survival of ICU Patients

Issued: Fri 09/25, Due: Sun 10/11 at 11:59pm

With the rapid advancement of technology, hospitals are creating and collecting vast amounts of data about their patients. These data, stored in the electronic health record, have the potential to impact clinical care in a variety of ways, e.g., in identifying patients at greatest risk of an adverse outcome, in predicting the efficacy of a particular drug, or in matching patients with the best treatments.

You will be working with a subset of data collected from patients admitted to an intensive care unit at the Beth Israel Deaconess Medical Center in Boston. You will develop classification models to identify patients at high risk of in-hospital mortality. These tools could help clinicians target interventions, facilitate better management, and improve patient outcomes. **We emphasize that these data represent real patients;** they have been made available for research and educational purposes by [PhysioNet.org](https://physionet.org).

Submission Instructions: The homework is due on Gradescope in **two** parts.

- **Upload PDF to HW3-Written:** Your submission may be typed or handwritten, and pages must be tagged with relevant parts on Gradescope. To help you navigate different sub-questions of the homework, we've **highlighted question parts** that should be included in the write-up.
- **Submit your code and challenge predictions to HW3-Code&Challenge:** Your submission must include the following files: hw3_main.py, hw3_challenge.py, challenge.csv. Questions marked with ✂ on the left margin are graded by a Python autograder. Upload your predictions for the held-out data in a single csv. You may submit multiple times; only the last submission will be considered. **The leaderboard is disabled for the duration of the homework.** Please also include the following **SIGNED** honor code statement:

THIS HOMEWORK IS MY OWN WORK, WRITTEN WITHOUT COPYING FROM OTHER STUDENTS
OR DIRECTLY FROM LARGE LANGUAGE MODELS SUCH AS CHATGPT.

Any collaboration or external resources have been properly acknowledged.

<add details>

/* Your_Name_Here */

Model Assessment and Model Selection

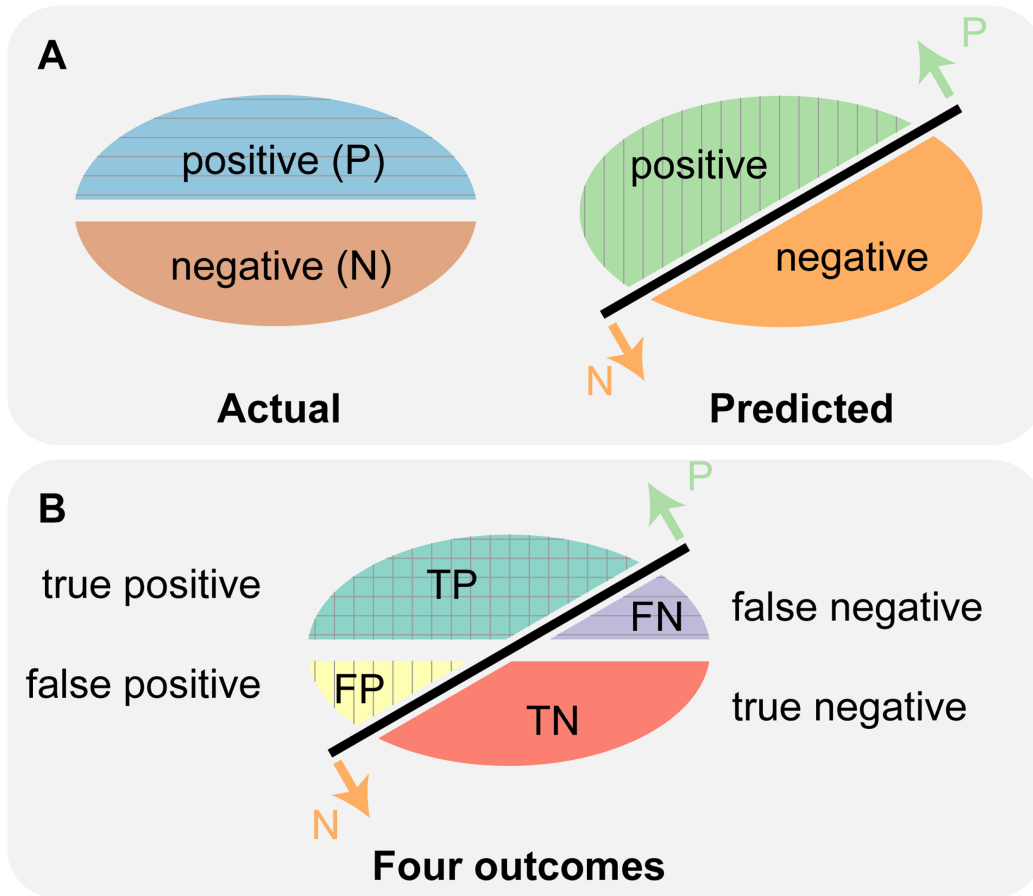
- **Model assessment:** evaluating a model's performance
- **Model selection:** selecting the proper level of flexibility for a model (e.g. regularization strength in logistic regression)

Model Assessment

- Performance Metrics:
 - Classification: accuracy, precision, recall, sensitivity, specificity, true positive rate, false positive rate, F1-score, AUROC, AUPRC, calibration*
 - Regression: MSE, RMSE, MAE, MBE
- Process:
 - training/test split (holdout)
 - K-fold cross-validation, Monte Carlo cross validation
 - bootstrap confidence intervals (bootstrap CIs)*

*: you're not responsible to know for the exam

Recap: Classification Metrics



$(TP+TN)/N$ accuracy

$TP/(TP+FN)$ true positive rate (TPR), sensitivity, recall

$TN/(TN+FP)$ true negative rate (TNR), specificity

$FP/(TN+FP)$ false positive rate (FPR), $1 - \text{specificity}$

$FN/(TP+FN)$ false negative rate (FNR), $1 - \text{sensitivity}$

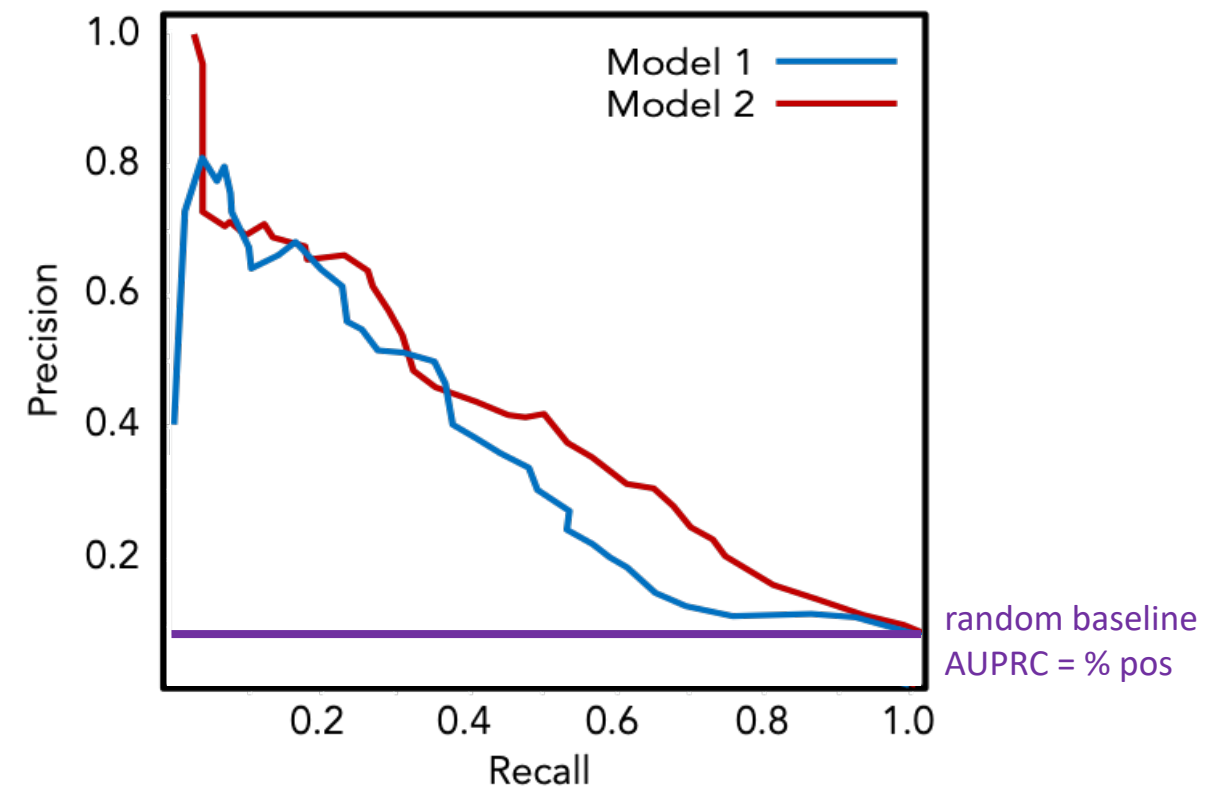
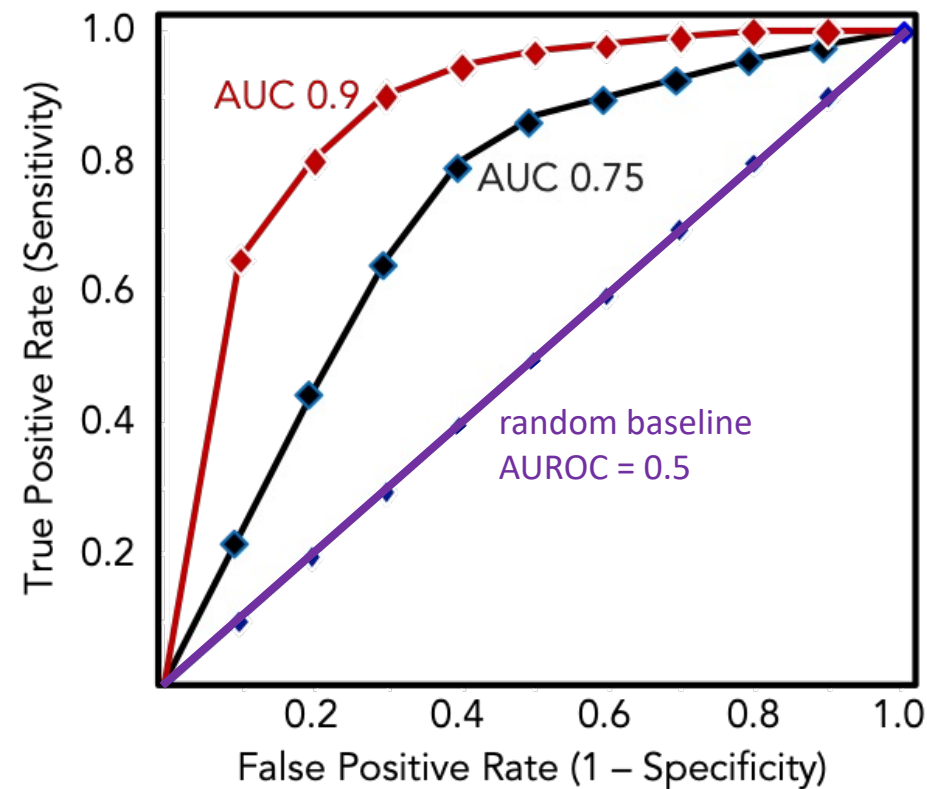
$TP/(TP+FP)$ precision, positive predictive value (PPV)

Can we optimize all metrics at the same time?

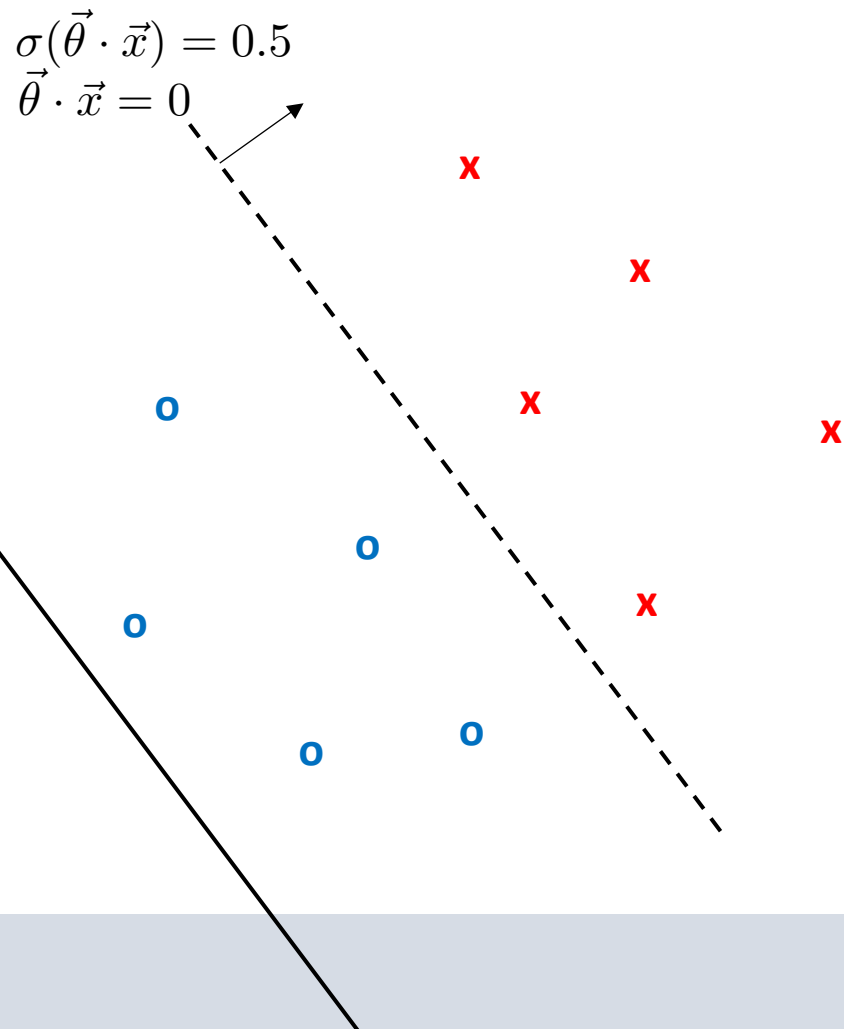
Composite metrics: e.g. balanced accuracy, F-scores

Recap: AUROC and AUPRC

Considers all discrimination thresholds using classifiers continuous output $\vec{\theta} \cdot \vec{x}$ or $\sigma(\vec{\theta} \cdot \vec{x})$



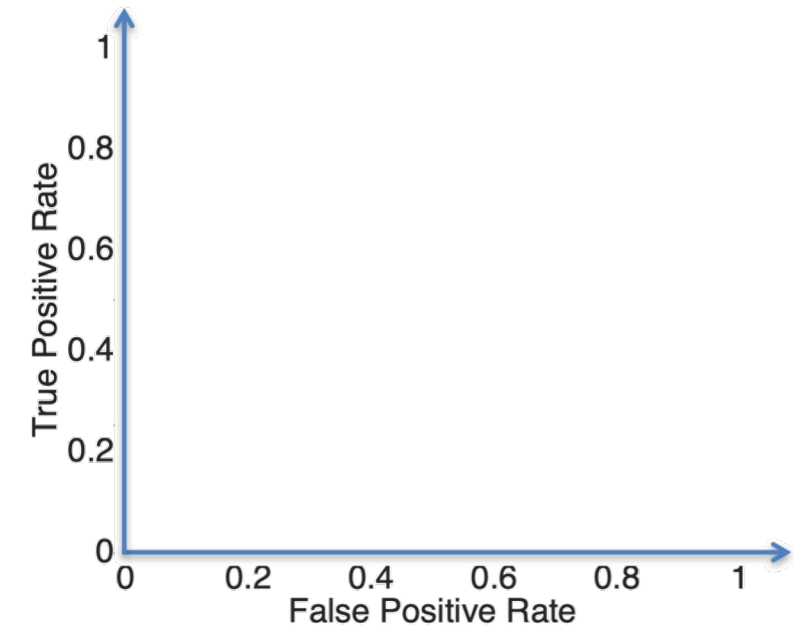
ROC curve: How to get multiple points?



$\hat{y}^{(1)} = 90\%$	$y^{(1)} = +1$
$\hat{y}^{(2)} = 80\%$	$y^{(2)} = +1$
$\hat{y}^{(3)} = 70\%$	$y^{(3)} = -1$
$\hat{y}^{(4)} = 60\%$	$y^{(4)} = +1$
$\hat{y}^{(5)} = 40\%$	$y^{(5)} = -1$
$\hat{y}^{(6)} = 30\%$	$y^{(6)} = -1$
$\hat{y}^{(7)} = 20\%$	$y^{(7)} = +1$
$\hat{y}^{(8)} = 10\%$	$y^{(8)} = -1$

Pos.
Neg.

True Positive Rate: $TP/(TP+FN)$
False Positive Rate: $FP/(TN+FP)$



Metrics: sklearn implementation

<code>confusion_matrix (y_true, y_pred, *[, ...])</code>	Compute confusion matrix to evaluate the accuracy of a classification.
<code>classification_report (y_true, y_pred, *[, ...])</code>	Build a text report showing the main classification metrics.
<code>precision_score (y_true, y_pred, *[, labels, ...])</code>	Compute the precision.
<code>recall_score (y_true, y_pred, *[, labels, ...])</code>	Compute the recall.
<code>f1_score (y_true, y_pred, *[, labels, ...])</code>	Compute the F1 score, also known as balanced F-score or F-measure.

<code>precision_recall_curve (y_true[, y_score, ...])</code>	Compute precision-recall pairs for different probability thresholds.
<code>roc_curve (y_true, y_score, *[, pos_label, ...])</code>	Compute Receiver operating characteristic (ROC).
<code>roc_auc_score (y_true, y_score, *[, average, ...])</code>	Compute Area Under the Receiver Operating Characteristic Curve (ROC AUC) from prediction scores.

`y_pred = clf.predict(X)`

Predicted Class $\{-1, +1\}$

One threshold

`y_prob = clf.predict_proba(X)[: ,1]`

`y_score = clf.decision_function(X)`

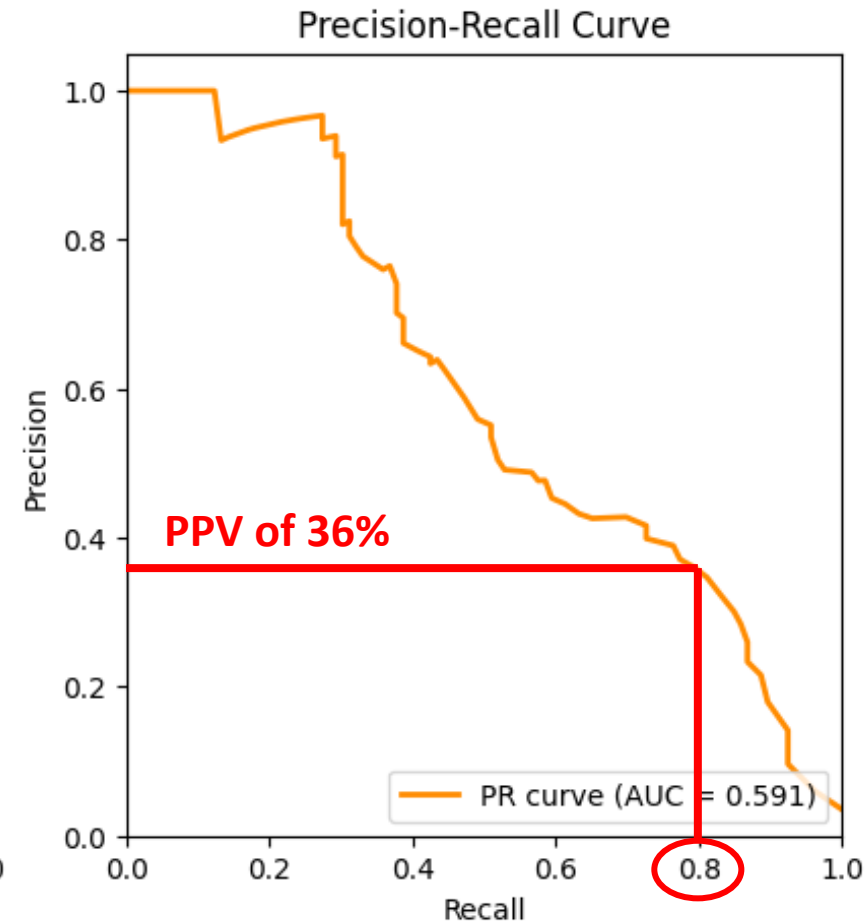
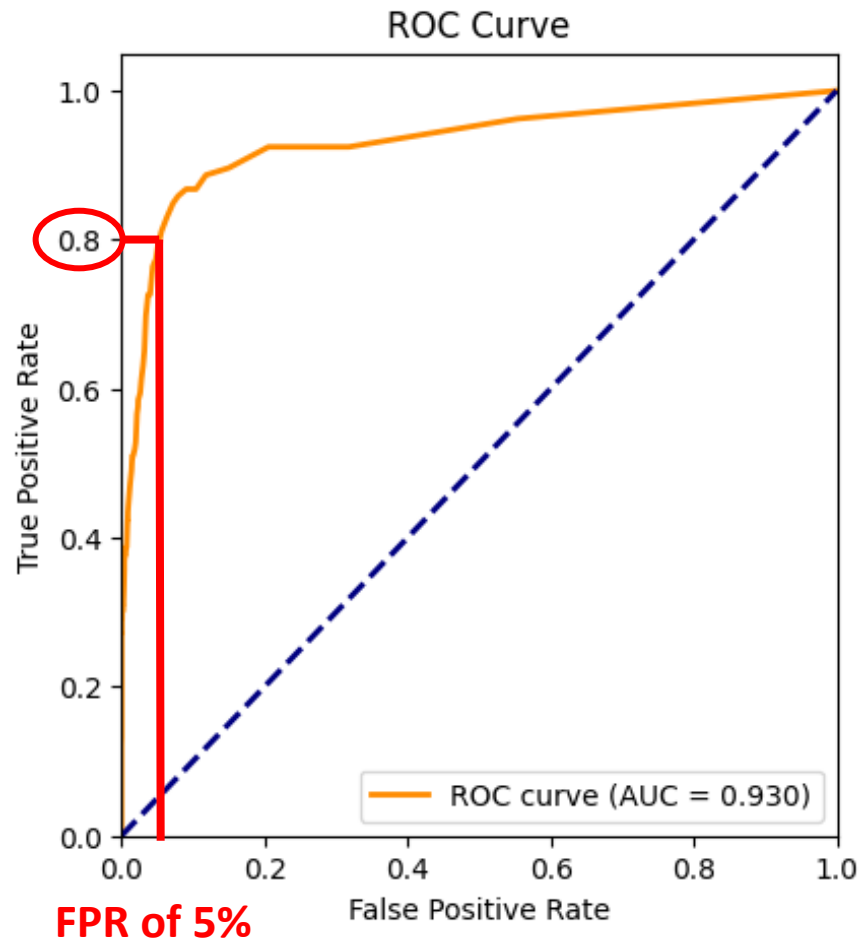
Predicted Class Probability $\sigma(\vec{\theta} \cdot \vec{x})$

Or continuous output $\vec{\theta} \cdot \vec{x}$

All possible thresholds

Selecting a Threshold

e.g. “I want a minimum sensitivity of 80%”
→ makes sure we catch 80% of the positive cases



*Classifier Probability Calibration

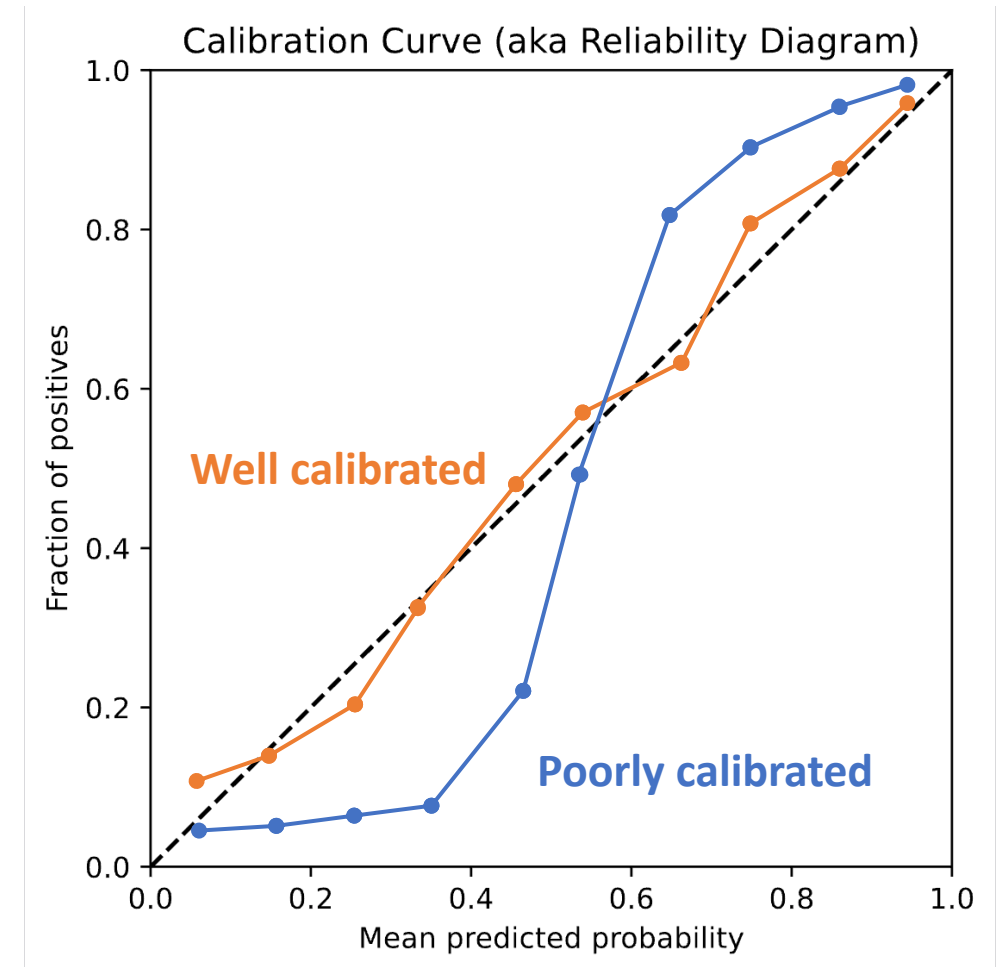
When a model produce a probability of positive class, is that number actually a meaningful probability?

$$h(\vec{x}; \vec{\theta}) = \sigma(\vec{\theta} \cdot \vec{x})$$

If my model predicts 90% for a set of examples, does that mean 90% of those examples have label +1?

A prediction of “90% chance of spam” should be spam 90% of the time.

<https://scikit-learn.org/stable/modules/calibration.html>



Metrics for Multiple Classes

- Accuracy:

$$ACC = \frac{TP1 + TP2 + TP3}{\text{Total}}$$

- Macro-average precision:

$$PRE_{\text{macro}} = \frac{PRE_1 + PRE_2 + PRE_3}{3}$$

- Micro-average precision:

$$PRE_{\text{micro}} = \frac{TP1 + TP2 + TP3}{TP1 + TP2 + TP3 + FP1 + FP2 + FP3}$$

(micro-average precision = micro-recall = accuracy)

	Predicted Class 1	Predicted Class 2	Predicted Class 3
Actual Class 1	TP1		
Actual Class 2		TP2	
Actual Class 3			TP3

Regression Metrics

Given $\hat{y} = f(\vec{x}; \vec{\theta})$ and $D = \{\vec{x}^{(i)}, y^{(i)}\}_{i=1}^N$ $\vec{x}^{(i)} \in \mathbb{R}^d$ $y^{(i)} \in \mathbb{R}$

Mean squared error

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

Mean absolute error

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N \left| \hat{y}^{(i)} - y^{(i)} \right|$$

Root mean squared error

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N \left(\hat{y}^{(i)} - y^{(i)} \right)^2}$$

Mean bias error

$$\text{MBE} = \frac{1}{N} \sum_{i=1}^N \left(\hat{y}^{(i)} - y^{(i)} \right)$$

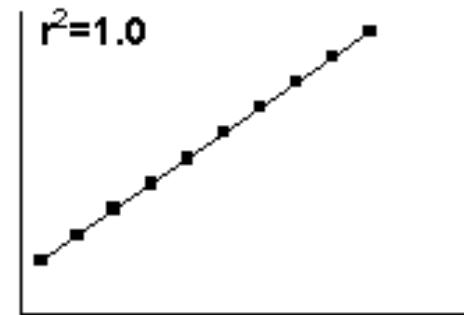
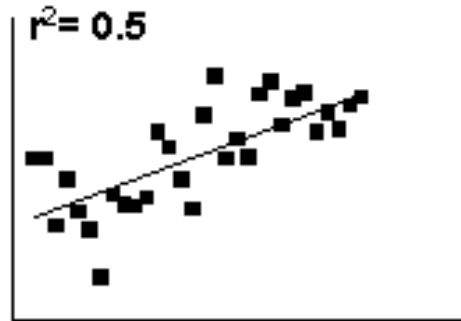
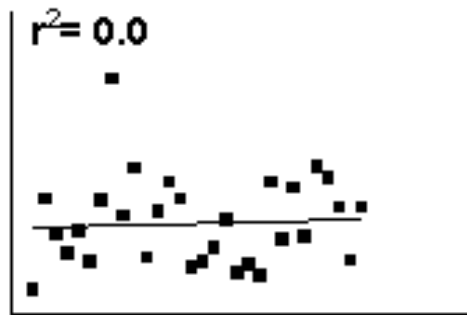
For each metric, think about the following:

- What is its range (minimum and maximum possible score)?
- What's the “perfect score”?

Measure of Fit: R^2

- “Goodness” of fit measure
- Interpretation: The proportion of variability in y explained by the model
- Always lies between 0 and 1

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

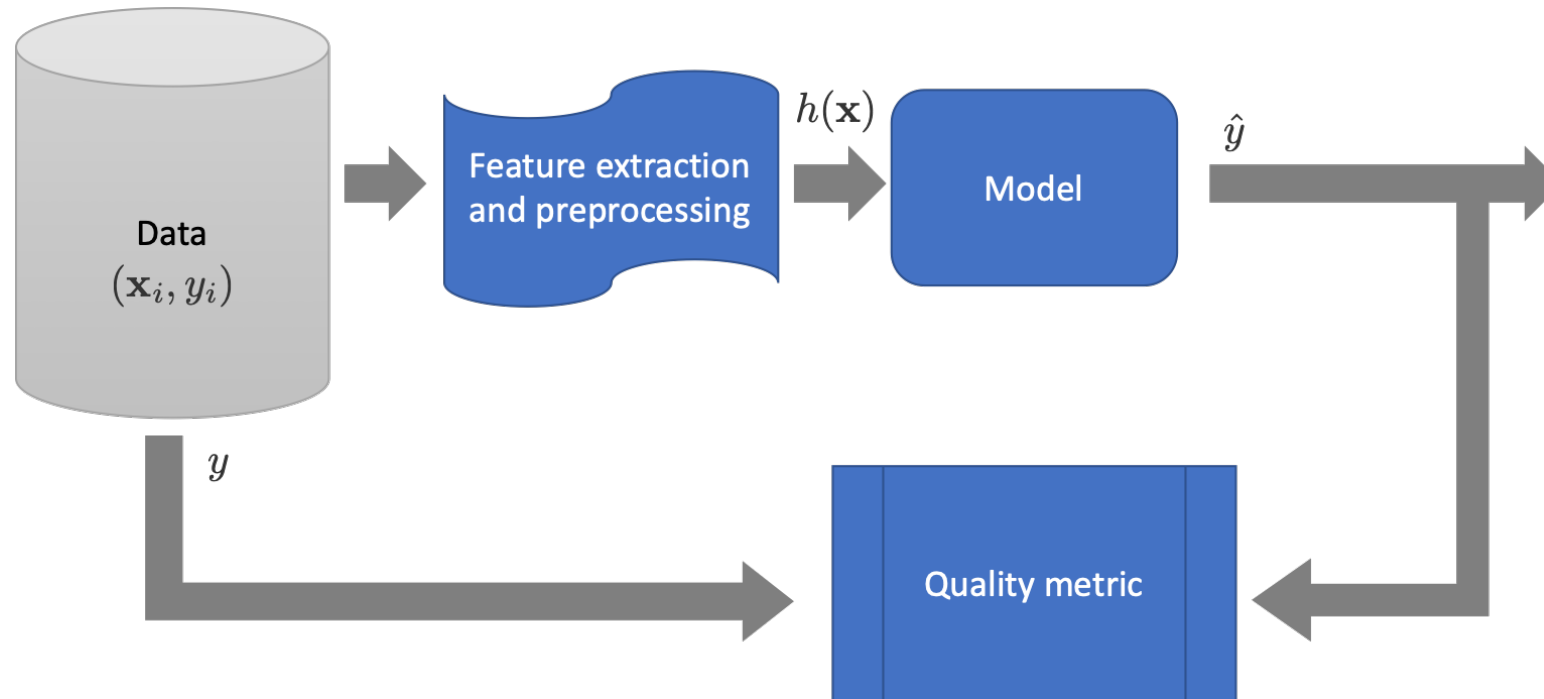


Model Assessment

- Performance Metrics:
 - Classification: accuracy, precision, recall, sensitivity, specificity, true positive rate, false positive rate, F1-score, AUROC, AUPRC, calibration*
 - Regression: MSE, RMSE, MAE*, MBE*
- Process:
 - training/test split (holdout)
 - K-fold cross-validation, Monte Carlo cross validation
 - bootstrap confidence intervals (bootstrap CIs)*

*: you're not responsible to know for the exam

Model Assessment

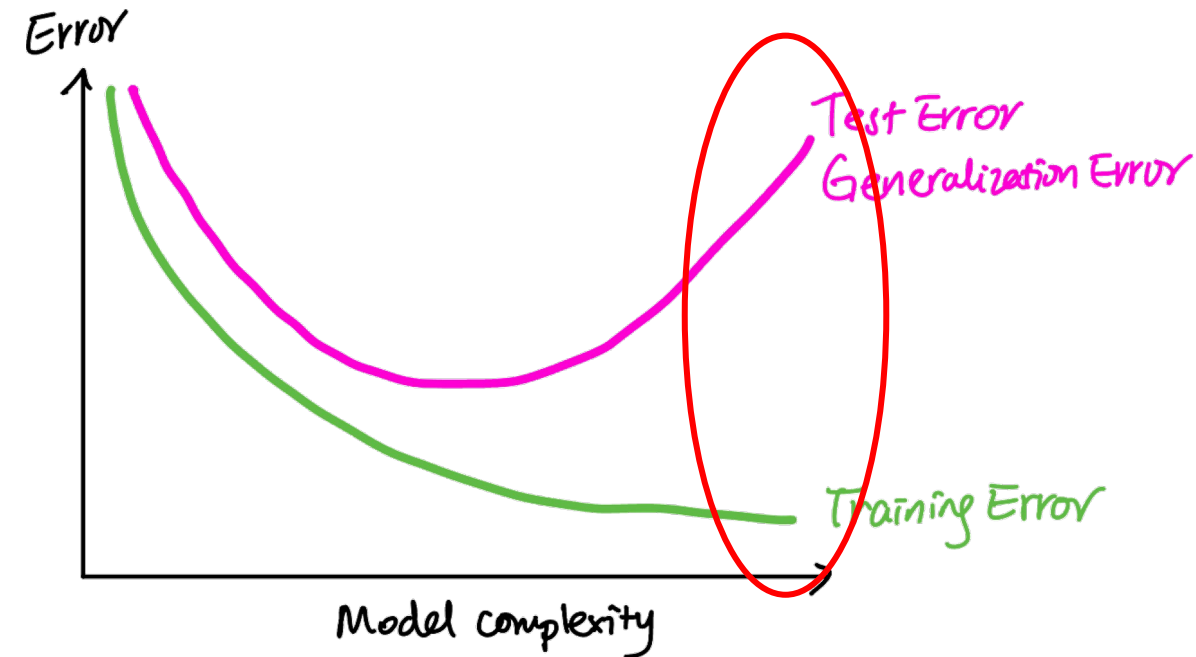


Use all the data to train the model and report the performance on all the data?

The pitfall of training error

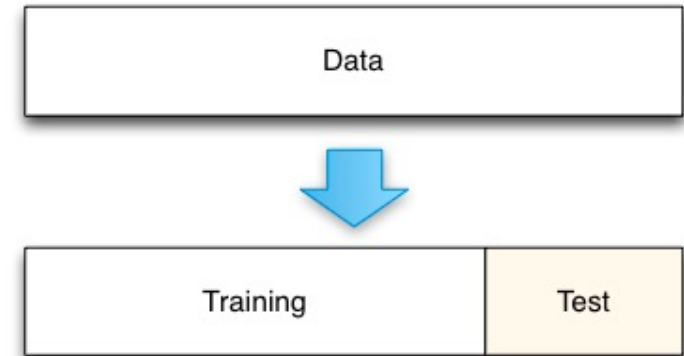
- When the model is overly complex, training error can be close to 0%
 - Lookup table
 - Kernelized classifiers with no regularization (Lec #11)
 - High-degree polynomial

Model “memorizes” the training data
But does not generalize to new data!



Holdout: Forming a test set

- Hold out some data (i.e., test data) that are not used for training the model
- Proxy for “everything you might see”



<http://scott.fortmann-roe.com/docs/MeasuringError.html>

Train/test split and Test error

	Date	Title	Budget	DomesticTotalGross	Director	Rating	Runtime
0	2013-11-22	The Hunger Games: Catching Fire	130000000	424668047	Francis Lawrence	PG-13	146
1	2013-05-03	Iron Man 3	200000000	409013994	Shane Black	PG-13	129
2	2013-11-22	Frozen	150000000	400738009	Chris BuckJennifer Lee	PG	108
3	2013-07-03	Despicable Me 2	76000000	368061265	Pierre CoffinChris Renaud	PG	98
4	2013-06-14	Man of Steel	225000000	291045518	Zack Snyder	PG-13	143
5	2013-10-04	Gravity	100000000	274092705	Alfonso Cuaron	PG-13	91
6	2013-06-21	Monsters University	NaN	268492764	Dan Scanlon	G	107
7	2013-12-13	The Hobbit: The Desolation of Smaug	NaN	258366855	Peter Jackson	PG-13	161
8	2013-05-24	Fast & Furious 6	160000000	238679850	Justin Lin	PG-13	130
9	2013-03-08	Oz The Great and Powerful	215000000	234911825	Sam Raimi	PG	127
10	2013-05-16	Star Trek Into Darkness	190000000	228778661	J.J. Abrams	PG-13	123
11	2013-11-08	Thor: The Dark World	170000000	206362140	Alan Taylor	PG-13	120
12	2013-06-21	World War Z	190000000	202359711	Marc Forster	PG-13	116
13	2013-03-22	The Croods	135000000	187168425	Kirk De MiccoChris Sanders	PG	98
14	2013-06-28	The Heat	43000000	159582188	Paul Feig	R	117
15	2013-08-07	We're the Millers	37000000	150394119	Rawson Marshall Thurber	R	110
16	2013-12-13	American Hustle	40000000	150117807	David O. Russell	R	138
17	2013-05-10	The Great Gatsby	105000000	144840419	Baz Luhrmann	PG-13	143

Training Data

X_{train}
 Y_{train}

`SVC().fit( $X_{train}$ ,  $Y_{train}$ )` → model

Test Data

X_{test}

model.predict(X_{test}) → $Y_{predict}$

`error_metric( $Y_{test}$ ,  $Y_{predict}$ )` → test error

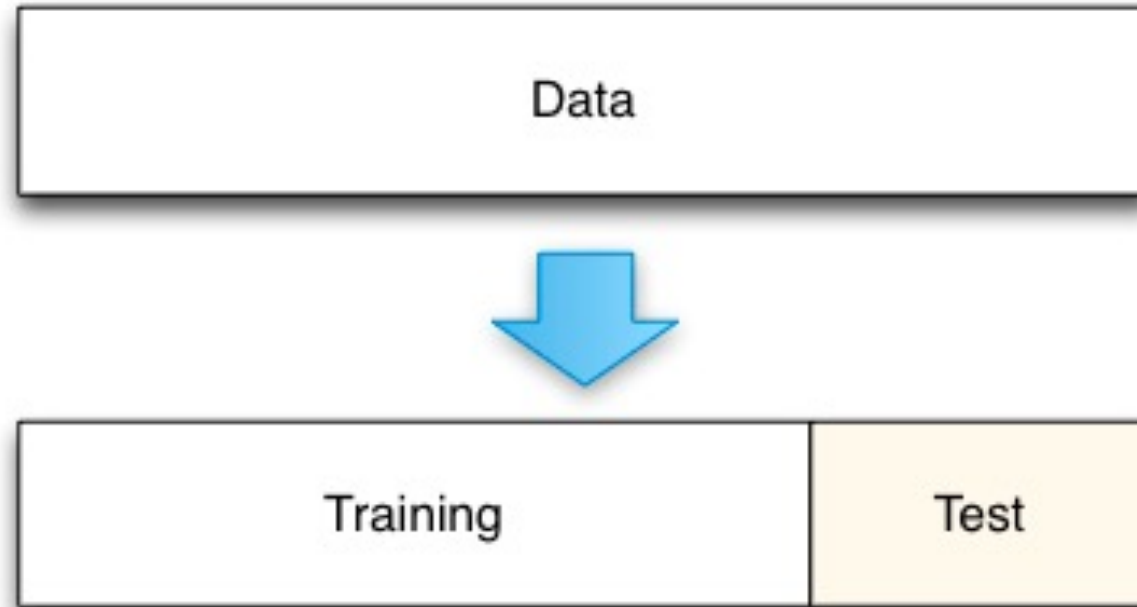
What should the percentage be?

Train/Test Splits

- Too few for training —> unable to properly learn from the data
- Too few for testing —> bad approximation of the true error
- Rule of thumb: enough test samples to form a reasonable estimate

Common split size is 70%-30% or 80%-20%

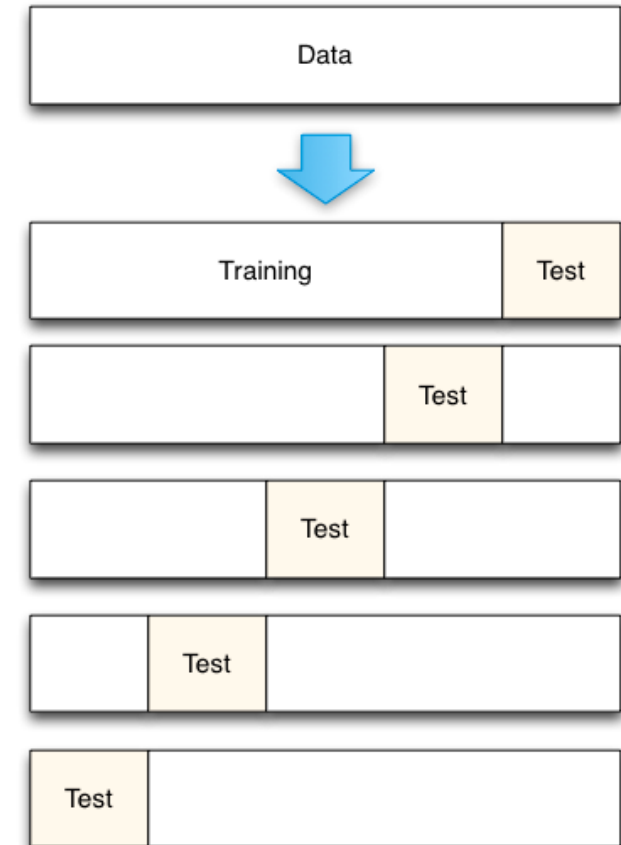
Train/Test Splits



What to do when there isn't enough data? Test data wasted?

K-fold Cross Validation

- Use all the data to train / test (but don't use all data to train at a time)
- Procedure:
 - Split the data into K parts or “folds”
 - Train on all but the k -th part and test/validate on the k -th part
 - Repeat for each k
 - Report average over K experiments



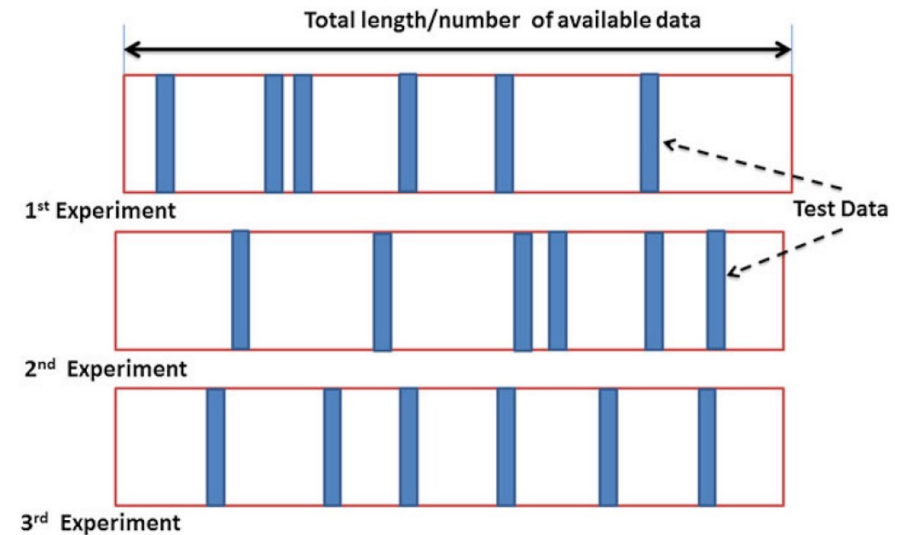
Common values of K

- $K = 2$ (two-fold cross validation)
- $K = 5, 10$ (5-fold, 10-fold cross validation) – common practice
- $K = N$ (leave-one-out cross validation or LOOCV)

Selection is based on how much data you have

Monte-Carlo Cross Validation

- AKA repeated random sub-sampling
 - Randomly select (without replacement) some fraction of your data to form training set
 - Assign remaining to test set
 - Repeat multiple times with different partitions





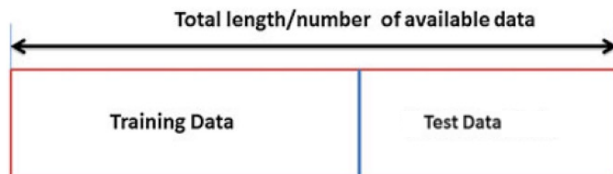
Group Activity

How many possible partitions do we see?

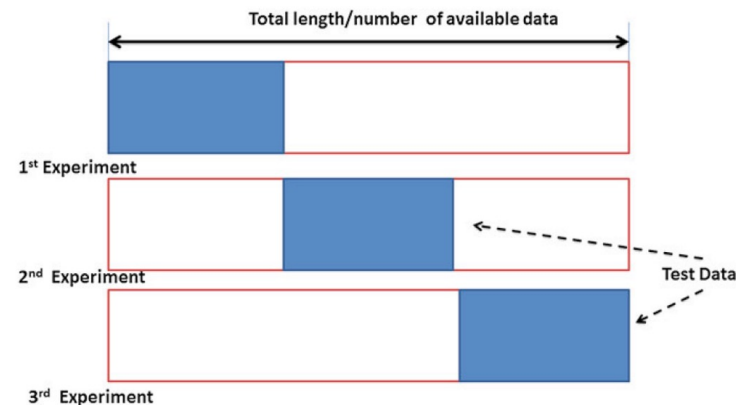
How many models do we need to train?

What's the “final” model?

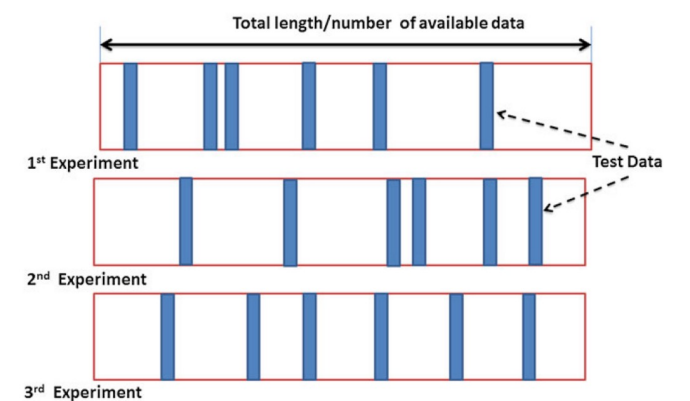
Holdout



K-fold Cross-Validation



Monte-Carlo Cross-Validation



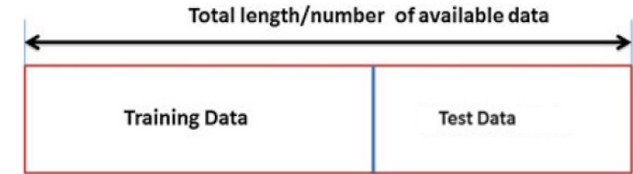
How many possible partitions do we see?

How many models do we need to train?

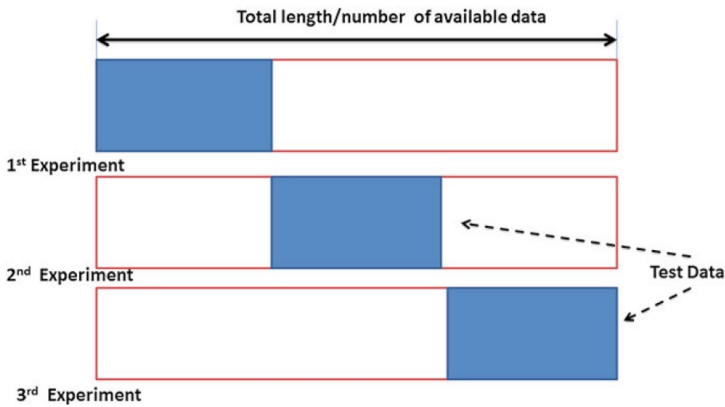
What’s the “final model”?

How to get error bars?

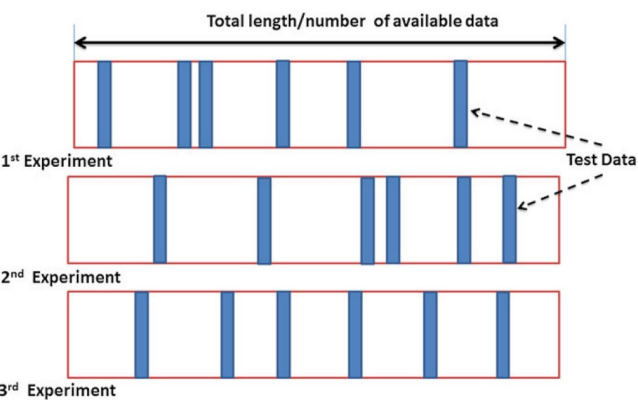
Holdout



K-fold Cross-Validation

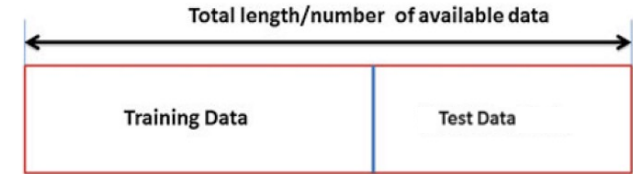


Monte-Carlo Cross-Validation

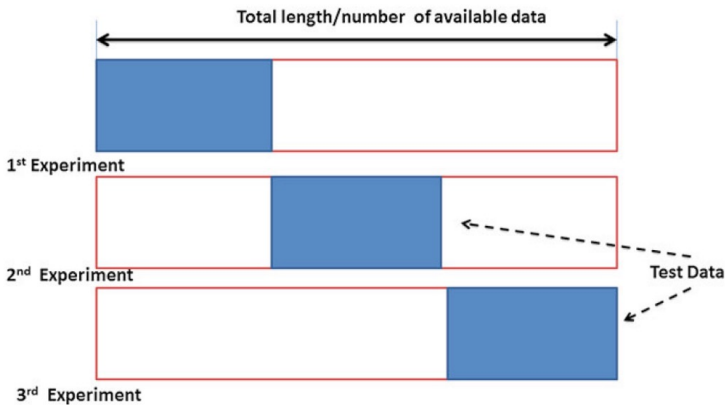


How many possible partitions do we see?	How many models do we need to train?	What's the "final model"?	How to get error bars?
1	1	The model	Bootstrap* the test set
K	K	Average (?)	Over K folds
as many as possible	if repeat 1000 times, we train 1000 models	Average (?)	Over 1000 repetitions

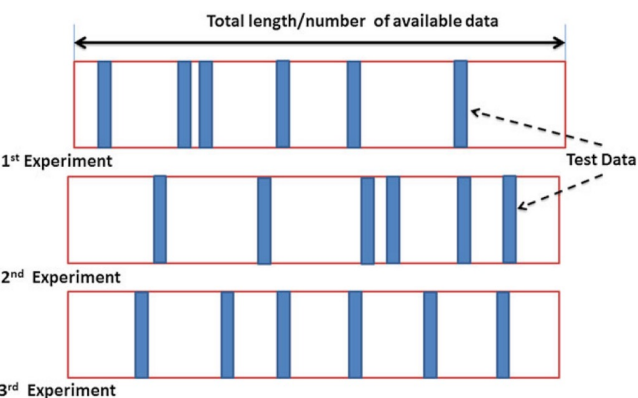
Holdout



K-fold Cross-Validation



Monte-Carlo Cross-Validation



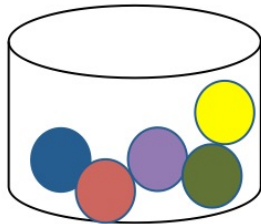
*Bootstrapped Confidence Intervals

- We want to know how performance is likely to vary
- In many settings, we have only a single test set
- Solution: bootstrap – given a test set of size N , sample with replacement N times to create a new test set, calculate the metric of interest on this new sample and repeat

* Bootstrapped Confidence Intervals

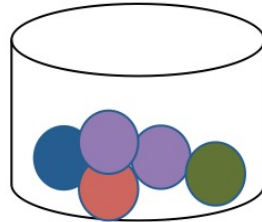
The Bootstrap (Efron 1979)

Original Sample

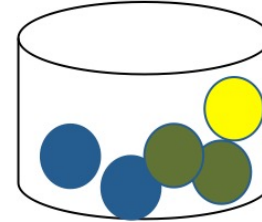


Sample **with replacement** 1000 times

BtS_1

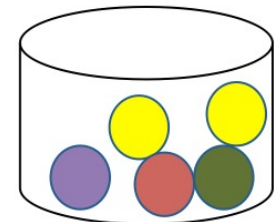


BtS_2

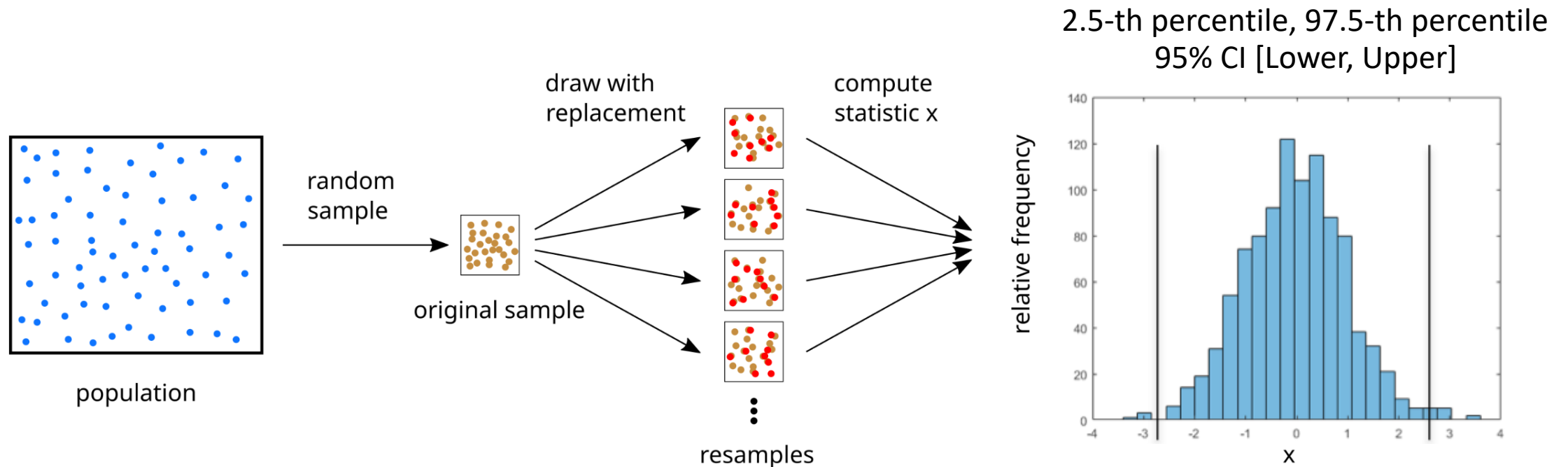


...

BtS_{1000}



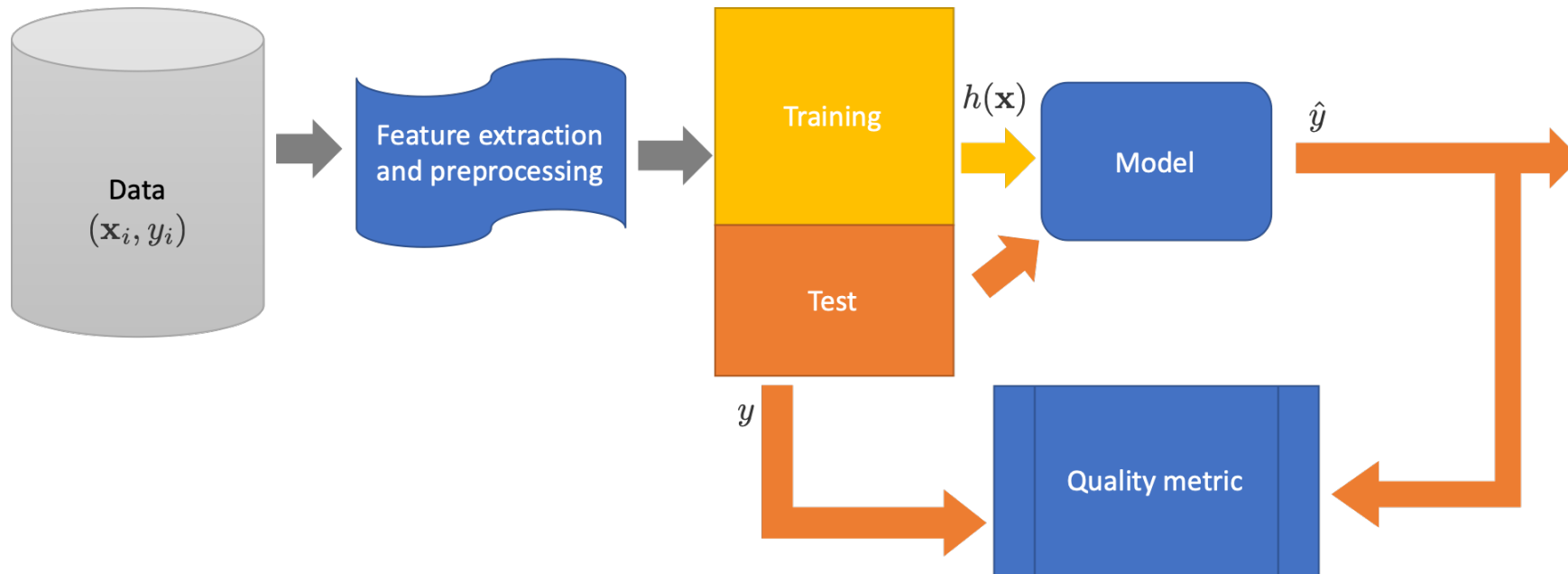
* Bootstrapped Confidence Intervals



x doesn't have to be the average it could be any sample statistic (e.g., AUROC, accuracy, F1-score)

Model Assessment: The Process

- Holdout
- K-fold CV
- Monte-Carlo CV



Report the performance on the “test” data

Model Assessment and Model Selection

- **Model assessment:** evaluating a model's performance
- **Model selection:** selecting the proper level of flexibility for a model (e.g. regularization strength in logistic regression)

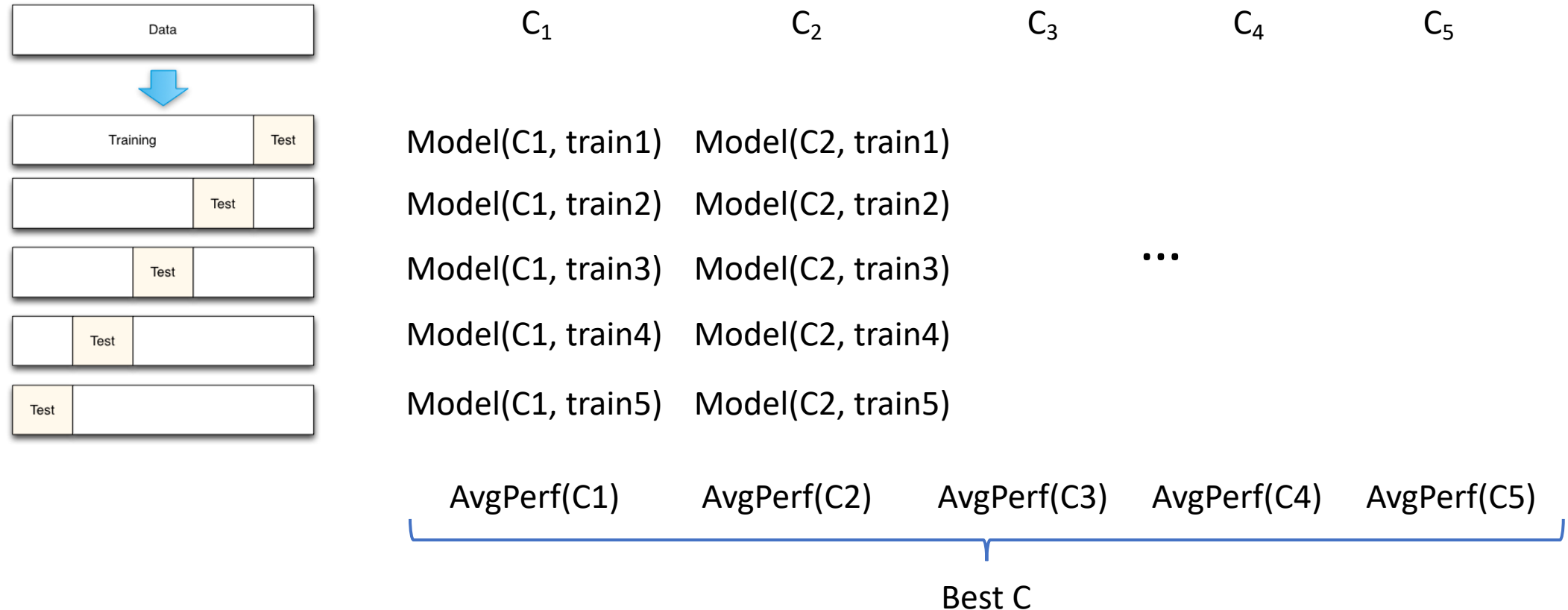
Model Selection

- Select **hyperparameters** of the model: “meta-optimization”
 - Regularization strength
 - Regularization type (L1 or L2)
 - Loss function (logistic regression or SVM)
 - Polynomial degree
 - Kernel type (linear vs RBF)
- Different from model **parameters**
 - Feature coefficients

How to select the best *hyperparameters*?

Simple, popular solution

K-Fold CV for Hyperparameter Selection

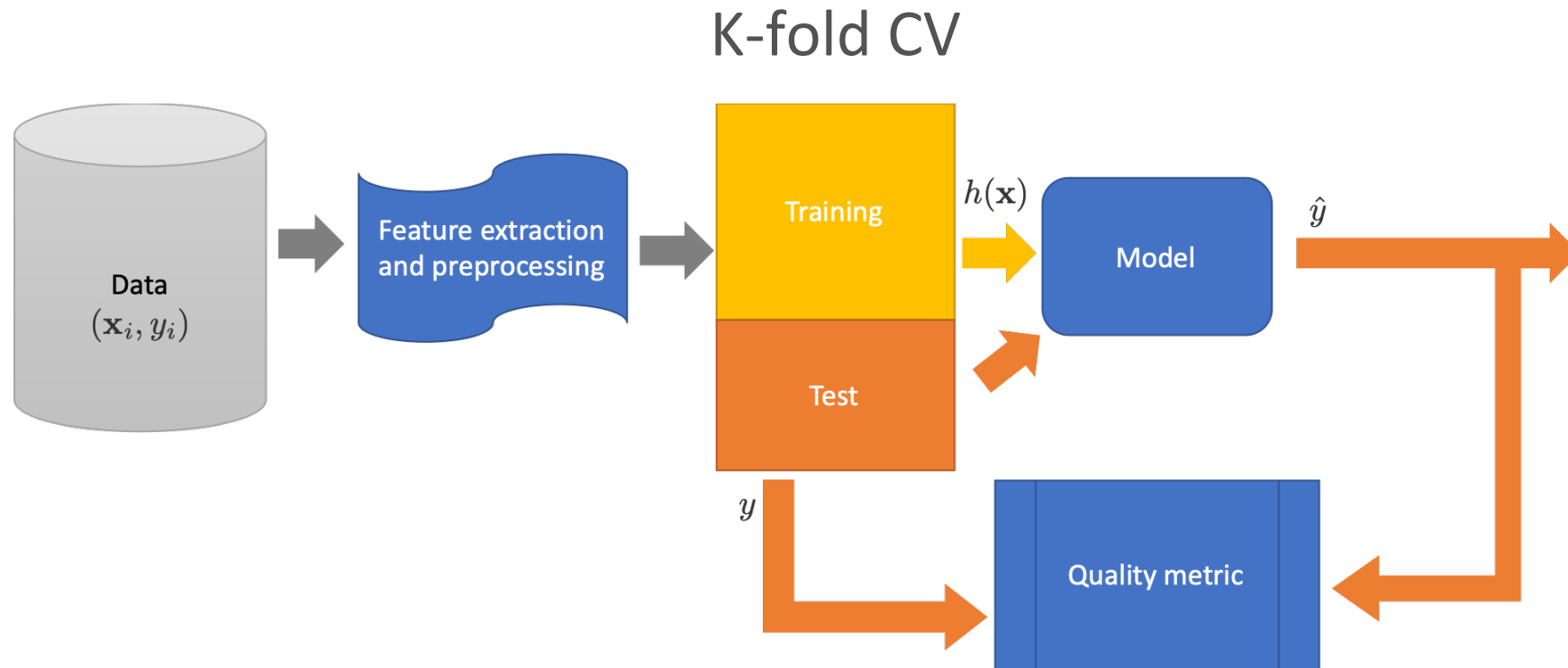


Model Assessment and Model Selection

- Model assessment: evaluating a model's performance
- Model selection: selecting the proper level of flexibility for a model (e.g. K for KNN, tree size for decision tree)

How to do both model selection and model assessment?

One Idea...

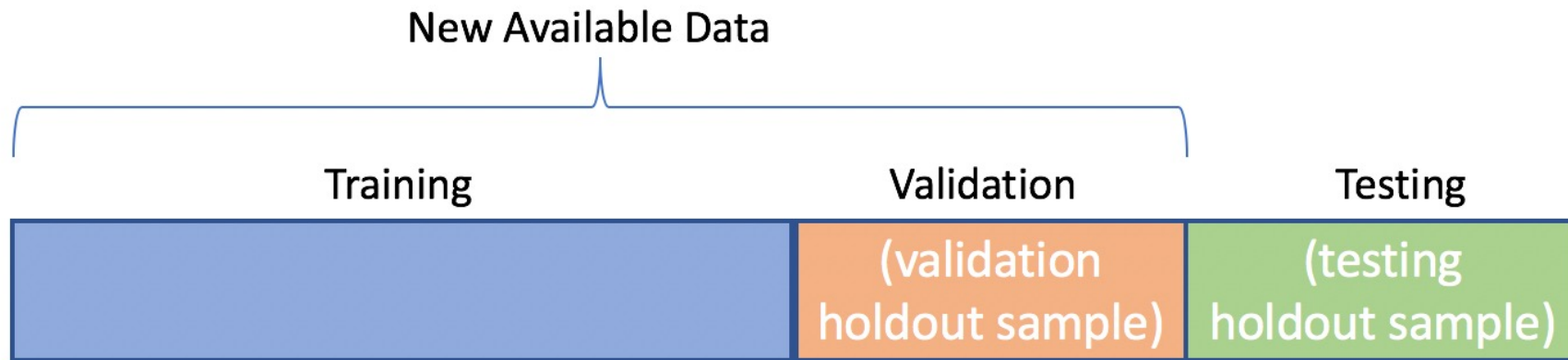
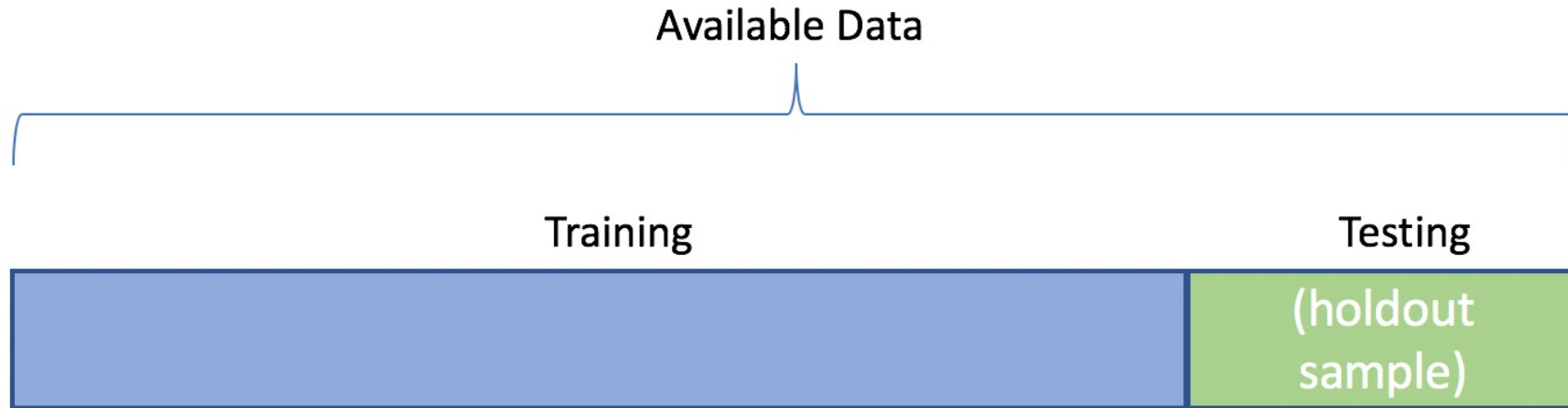


Tune the hyperparameters AND report the performance using cross validation -
Is this a good idea?

Assessment + Selection: Guidelines

- Don't use the same samples to choose optimal hyperparameters and to estimate test/generalization error
- Choice of methodology will depend on your problem and dataset

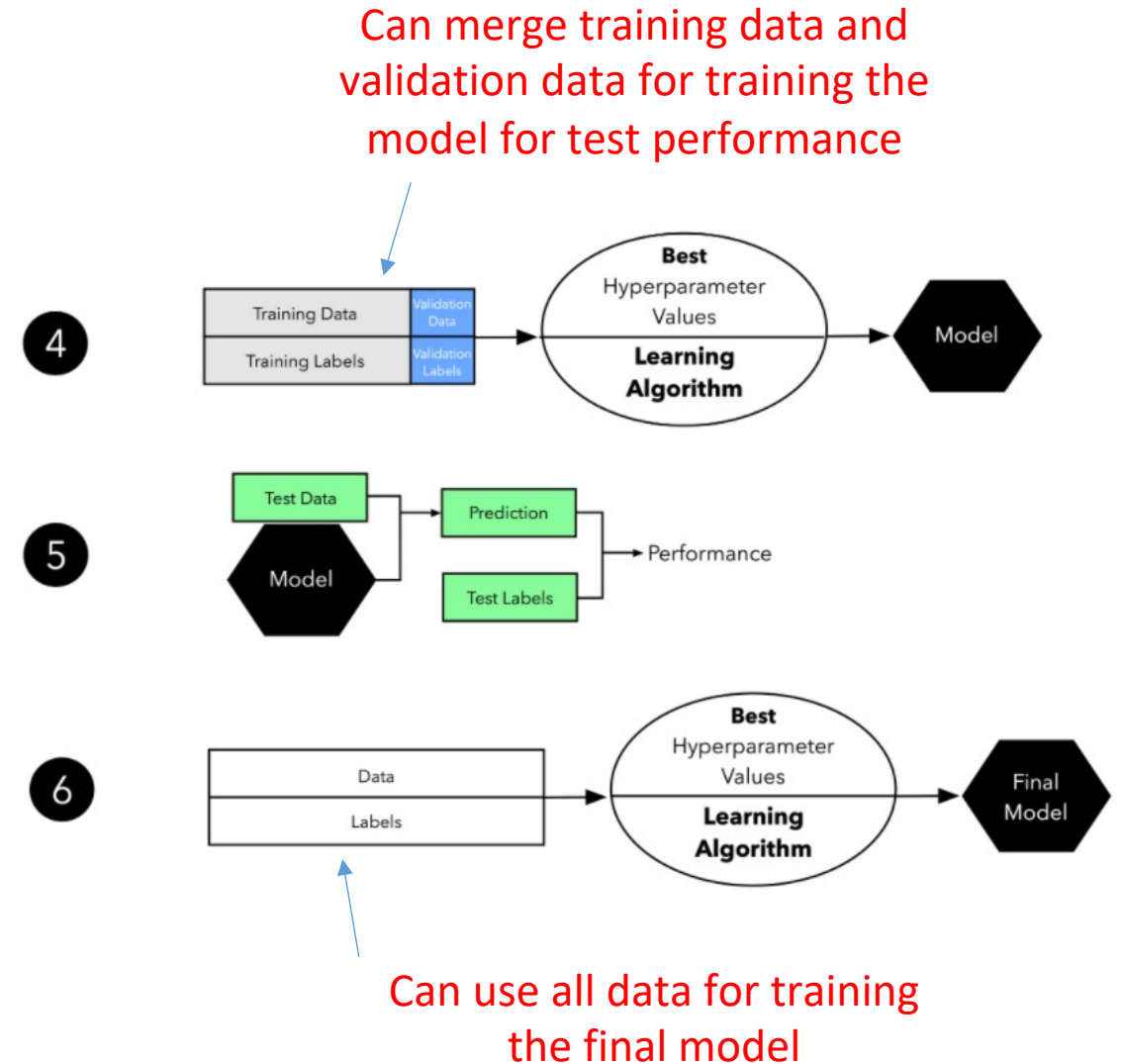
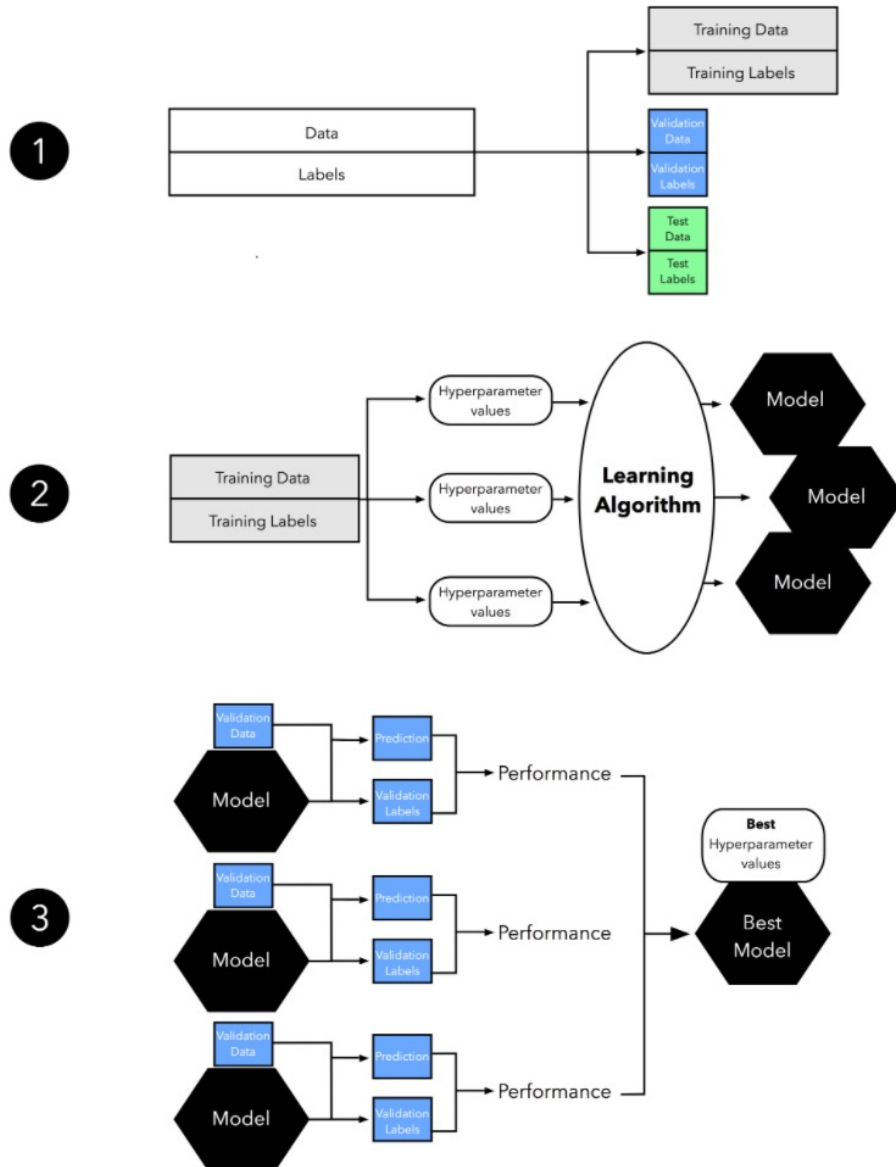
Three-way split



Validation data will
NOT be used for
training model

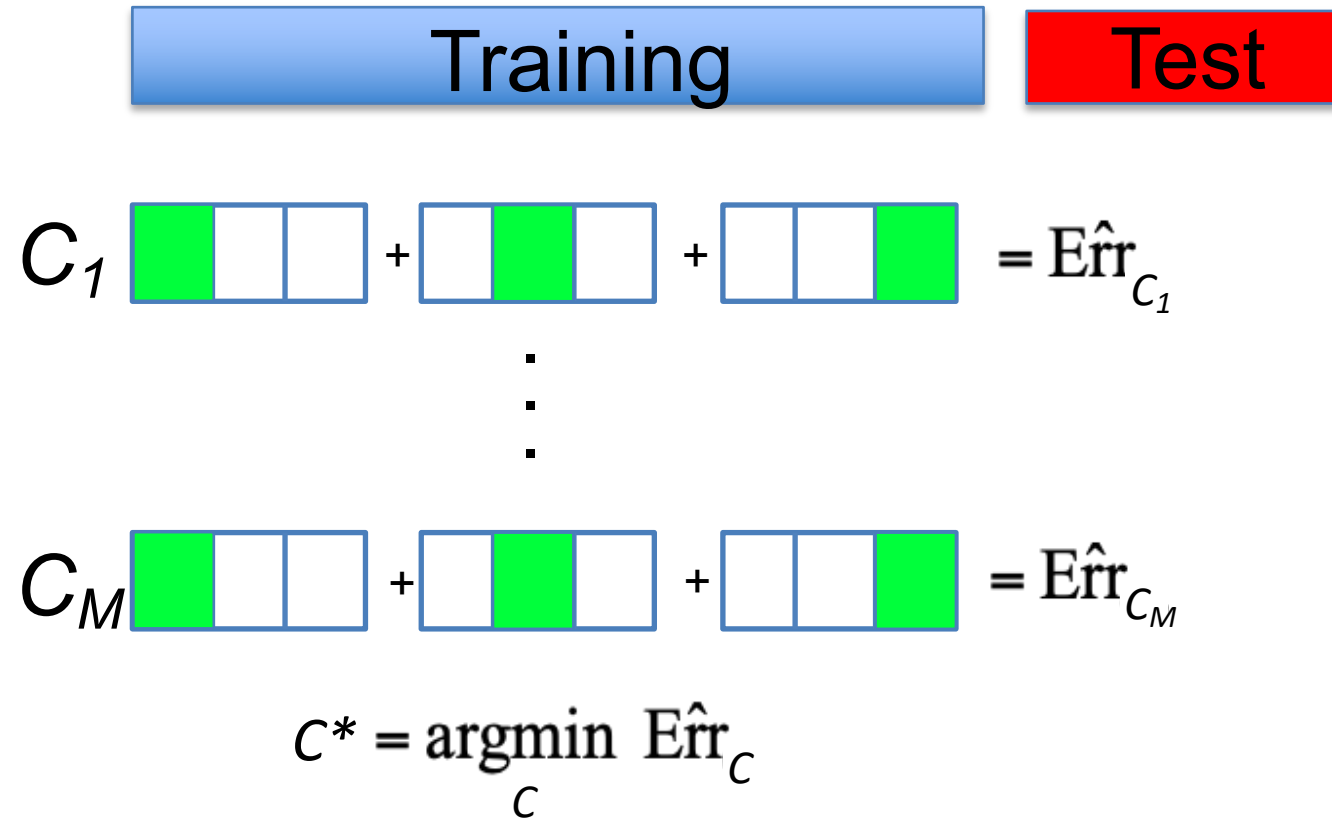
Test data will NOT be used
for training model or
tuning hyperparameters

Model selection and assessment: process (three-way split)

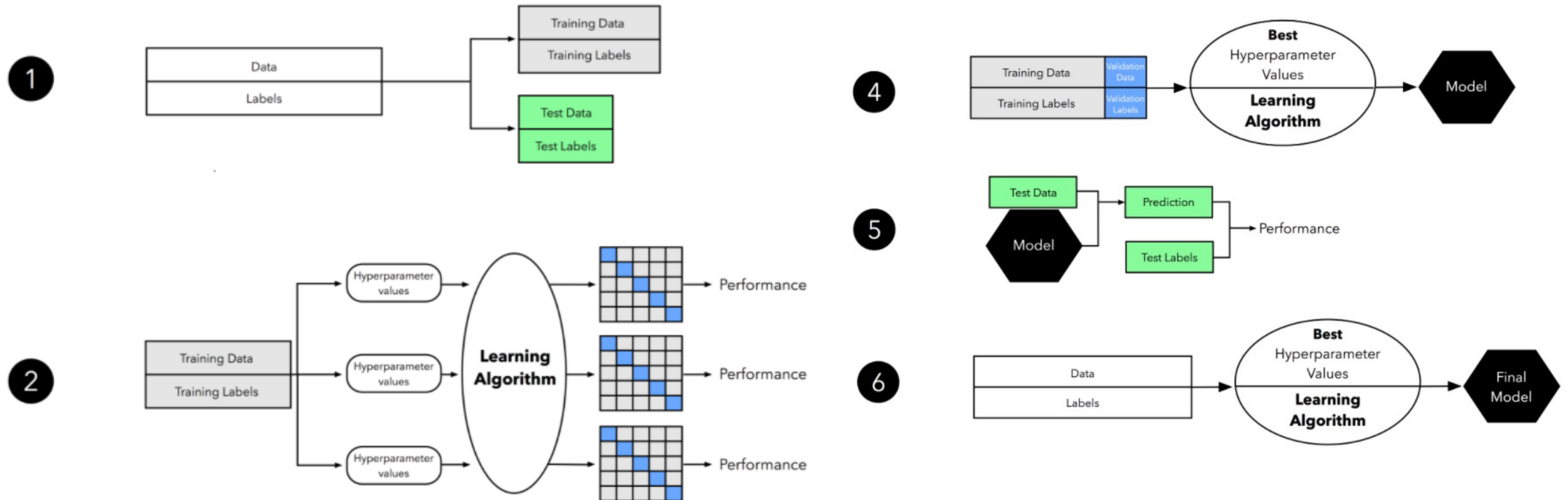


Holdout + K-Fold CV

- Holdout: Test dataset to assess the performance
- Training set: Use k-fold CV to find optimal hyperparameter
- Use optimal hyperparameters to train on the training data and assess on test



Model selection and assessment: process (holdout + CV)



Nested CV

- Outer k-fold loop: Assess the performance
- Inner k-fold loop: Choose the optimal hyperparameter



Model Selection + Assessment: sklearn

Most commonly used functions/classes: train_test_split, KFold, StratifiedKFold

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=1/9)
```

This gives an 80-10-10 train-val-test three-way split

```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5)
for train_idx, test_idx in skf.split(X, y):
    X_train, X_test = X[train_idx], X[test_idx]
    y_train, y_test = y[train_idx], y[test_idx]
```

This gives a 5-fold split in a for-loop
Stratify: maintains label proportions

Hyperparameter Search Space

- GridSearchCV: exhaustive search for best parameter values using cross validation
- RandomizedSearchCV: randomized search over parameters

For HW3 Q2 – implement your own grid search for cross validation!

```
def cv_performance(clf, X, y, k=5, metric='accuracy'):
    # TODO: Implement this function
    skf = StratifiedKFold(n_splits=k)
    scores = []
    # For each split in the k folds...
    for train, val in skf.split(X,y):
        # Split the data into training and validation sets...
        X_train, y_train, X_val, y_val = ???
        # Fit the data to the training data...
        ???
        # And test on the ith fold.
        score = ???
        scores.append(score)
    # Return the average performance across all
    return np.array(scores).mean()
```

```
def select_C(X, y, C_range=[], penalty='l2', k=5, metric='accuracy'):
    scores = []
    # Iterate over all of the given C range...
    for C in C_range:
        # Calculate the average performance on k-fold cross-validation
        clf = get_classifier(penalty=penalty, C=C)
        score = cv_performance(clf, X, y, k, metric)
        print("C: {:.6f} \t score: {:.4f}".format(C, score))
        scores.append((C, score))
    # Return the C value with the maximum score
    maxval = max(scores, key=lambda x: x[1])
    return maxval[0]
```

For HW3 Q3 – feel free to use sklearn grid/random search, read documentation to learn how to use them

```
from sklearn.model_selection import GridSearchCV
clf = GridSearchCV(
    LogisticRegression,
    {'C': [0.1, 1, 10, 100]},
    scoring='accuracy',
    cv=StratifiedKFold(n_splits=5),
)
```

In-Class Exercise #8

How many models are you training (for HW3)?

Q2 The ML Pipeline

0.2 Points

Read the following description of how someone has trained their machine learning model.

"To train a binary classification model, we used logistic regression with elastic net regularization. We first split the dataset into a training set and a test set. We performed 5-fold cross-validation on the training set, and used grid search to select the hyperparameters including L1 and L2 regularization strengths λ_1, λ_2 both $\in \{10^{-4}, 10^{-3}, \dots, 10^5\}$. The best hyperparameter setting (i.e. the values of λ_1 and λ_2 that led to the highest average cross-validation AUROC score) were then used to train a final model on the entire training set, and we report its performance on the test set."

What's the total number of models that were trained?

Q3 ML Pipeline - Comparison 1

0.2 Points

The above pipeline uses **grid search** for hyperparameter selection. What if we instead used **random search** (with 10 trials)?

What's the total number of models that were trained?

Q4 ML Pipeline - Comparison 2

0.1 Points

The above pipeline describes the "Holdout+CV" setup. If we instead used the "Nested CV" setup, do we need to train **fewer models** or **more models** than above?

- ☐ fewer
- ☐ more